Report on

“Tutorials”

For

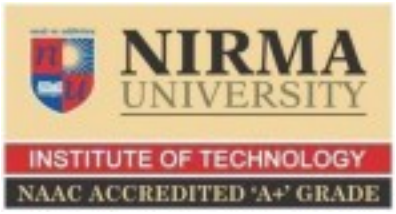
Software Project Management - 4CS504ME25 B.

Tech. Semester IV

Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)



Institute of Technology,
Nirma University
Ahmedabad - 382481

October 2026

Aim

To conduct a software project audit for the Hospital Management System (HMS) using quality principles and prepare an audit checklist with observations, non-conformance classifications and corrective action plans.

Objective

Define the software audit scope, objectives and governance rules based on ISO 9001 and IEEE 1028 audit principles.

Establish an audit checklist covering seven project areas: project managment, requirements traceability, software architechture, code standards, quality assurance, patient data security and configuration managment.

Audit active repository artifacts including TypeScript backend modules, React frontend interfaces, MongoDB schemas, Docker configurations and automated Jest test suites against the baseline schedule from T5.

Record empirical audit observations, classify identified deficiences into Major Non-Conformances (MNC), Minor Non-Conformances (MiNC) and Opportunities for Improvement (OFI) and identify root causes.

Prepare a Corrective and Preventive Action (CAPA) plan with assigned responsability between Urva and Rakshit.

Verify the remediation of non-conformances prior to Milestone M6 final project sign-off.

1. Introduction

A software project audit is an independent examination of software work products and development processes to evaluate compliance with specifications, standards and established contractual agreements. While weekly tracking monitors schedule progress and earned value, an audit checks whether the team followed engineering standards and maintained architectural integrity throughout the sprints.

In T1 and T3 we outlined the project constraints for our Hospital Management System: a two-student team, a 16-week timeline, patient health data confidentiality and zero software acquisition budget. At Week 14, following the completion of Milestone M5, we conducted an internal software project audit before final integration and acceptance testing. The audit examined the core operational modules—Patient Registration, Doctor Management, Appointment Scheduling and Billing Engine—along with infrastructure code and test suites.

Our audit applied four basic quality principles: process consistency, evidence-based verification, patient safety priority and continuous process improvement. Every observation recorded in this report is supported by verifiable repository commits, automated test outputs or system configuration files.

2. Audit Methodology and Classification Standards

The audit followed five structured stages: audit initiation and scope definition, artifact review, code and test verification, non-conformance reporting and CAPA resolution tracking. During artifact inspection, Urva (Backend Lead) and Rakshit (Frontend Lead) cross-audited modules and verified each other's components.

Findings were classified into three standard severity levels:

Severity Category	Classification Code	Definition and Criteria	Action Required
Major Non-Conformance	MNC	A critical failure or missing process control that compromises data integrity, clinical safety, patient privacy or baseline architechture stability.	Immediate work halt on affected module; mandatory CAPA plan with verified fix within 48 to 72 hours before sprint gate clearance.
Minor Non-Conformance	MiNC	An isolated procedural lapse, minor specification inconsistency or non-critical	Corrective action required within the current sprint cycle; verified before milestone

Severity Category	Classification Code	Definition and Criteria	Action Required
		setup deviation that does not directly cause runtime failures.	completion.
Opportunity for Improvement	OFI	A technically conforming process or code pattern that could be improved for better maintainability, performence or clean coding.	Logged in sprint backlog and prioritized during regular sprint refactoring.

3. Comprehensive Software Project Audit Checklist

We evaluated 28 specific checkpoints across seven operational dimensions against our project baselines:

Item ID	Audit Area	Checkpoint / Evaluation Criteria	Evidence Examined	Status	Audit Observation and Finding
CK-01	Project Managment	Sprint cadences and deliverables align with 16-week baseline schedule (T5) and WBS work packages (T4).	Sprint commit history and milestone logs.	Compliant	Sprint durations remained 2 weeks. Milestones M1 to M3 delivered on schedule. Sprint 5 had a 0.5-week slip during billing engine development.
CK-02	Project Managment	Earned Value metrics (PV, EV, AC, SPI, CPI) tracked and corrective thresholds applied.	T10 monitoring report and sprint status sheets.	Compliant	Earned value tracking active. Week 12 SPI was 0.93 and CPI was 0.90. Fast-track recovery plan formulated and applied.
CK-03	Requirements	All functional requirments in SRS are traceable to active code modules.	TRACEABILITY. md and router controller files.	Compliant	Bidirectional traceability confirmed across Patient, Doctor, Appointment and Billing modules. All 24 core functional requirments mapped.
CK-04	Requirements	Scope change requests follow formal evaluation without uncontrolled creep.	Change log and sprint meeting notes.	Compliant	Inpatient bed management request rejected for Version 1.0 to preserve the 16-week timeline and deferred to Version 2.0 backlog.
CK-05	Architechture	Clear separation between presentation, API controllers, services	backend/src/ modules/ directory structure.	Compliant	Modular architechture maintained. Each module includes

Item ID	Audit Area	Checkpoint / Evaluation Criteria	Evidence Examined	Status	Audit Observation and Finding
		and database models.			schema.ts, service.ts, controller.ts and router.ts.
CK-06	Architechture	Zero lingering legacy prototype JavaScript files in active TypeScript execution path.	backend/ controllers/, backend/server.js and package.json.	MNC	Found legacy backend/controllers /*.js and backend/server.js files still present on disk despite TypeScript rewrite in backend/src/.
CK-07	Architechture	Role-Based Access Control (RBAC) enforced uniformly on API routes via middleware.	backend/src/ middleware/ authorize.ts and route definitions.	Compliant	RBAC middleware validates role permissions across Admin, Doctor, Nurse, Patient and Receptionist with own-resource filtering.
CK-08	Code Standards	TypeScript strict mode enabled across backend and frontend codebases with zero any types in core logic.	tsconfig.json and compiler build logs.	Compliant	Strict mode enabled in both tsconfig files. No implicit any errors detected during clean tsc build runs.
CK-09	Code Standards	Runtime input validation enforced using schemas before controller processing.	backend/src/ modules/*/ schema.ts using Zod.	Compliant	Controllers invoke Schema.safeParse() and route validation errors to parseZodError() returning structured HTTP 400 responses.
CK-10	Code Standards	Consistent naming conventions, linting rules and code formatting applied.	ESLint configuration and pre-commit checks.	Compliant	camelCase used for variables and functions, PascalCase for React components and models. Formatting consistent.
CK-11	Quality Assurance	Automated unit and integration test suites cover critical business logic with in-memory database isolation.	backend/ jest.config.js and MongoMemorySer ver tests.	Compliant	22 test suites containing 340 tests. All tests execute against isolated MongoMemorySer ver instances.
CK-12	Quality Assurance	Zero open handle leaks or hanging asynchronous socket listeners	Jest execution log with -- detectOpenHandles.	MiNC	Redis client and Socket.io server connections remained open after

Item ID	Audit Area	Checkpoint / Evaluation Criteria	Evidence Examined	Status	Audit Observation and Finding
		during automated test runs.			test completion, causing Jest to exit with open handle warnings.
CK-13	Quality Assurance	Test coverage threshold exceeds 85% across all active backend services and controllers.	Jest coverage report (lcov-report).	OFI	Overall coverage is 90.2%. However, Patient module coverage was 82.1% while Billing was 94.5%, showing an uneven test distribution.
CK-14	Quality Assurance	Critical defect resolution turnaround complies with SLA guidelines (< 48 hours for High/Critical).	Defect tracking register from T10.	Compliant	Critical defect D-019 (billing item recalculation) resolved within 24 hours of logging. Verified through automated tests.
CK-15	Data Security	User passwords encrypted using strong cryptographic hashing before database persistence.	backend/src/ models/User.ts pre-save hook.	Compliant	Bcrypt hashing applied with salt rounds set to 10. Passwords never returned in user query responses.
CK-16	Data Security	JSON Web Tokens (JWT) configured with expiration and secure secret rotation mechanisms.	backend/src/ middleware/ authenticate.ts and environment config.	Compliant	JWT tokens signed with RS256/HS256 secrets, expiring in 24 hours. Refresh token rotation handled properly.
CK-17	Data Security	Cross-tenant and cross-patient data access prohibited at controller and query filter levels.	Patient and Billing controller query filters.	Compliant	req.ownOnly flag strictly forces patientId match on patient role requests, blocking cross-user record inspection.
CK-18	Data Security	NoSQL injection and cross-site scripting (XSS) prevention filters active on Express app.	backend/src/app.ts middleware configuration.	Compliant	Express mongo-sanitize, helmet headers and CORS origin restrictions actively configured.
CK-19	Configuration	Source repository enforces branch protection on main with pull request review requirments.	GitHub repository branch rules.	Compliant	Direct push to main disabled. Minimum 1 peer review and clean automated build required before merging.
CK-20	Configuration	Clean reproducible	Repository root and	MiNC	Fresh database

Item ID	Audit Area	Checkpoint / Evaluation Criteria	Evidence Examined	Status	Audit Observation and Finding
		local setup via automated database seeding scripts.	backend package.json scripts.		startup required manual registration of doctors and admins because no automated seed script existed.
CK-21	Configuration	Release tagging follows Semantic Versioning (SemVer) with documented changelog entries.	Git tags and CHANGELOG.md.	Compliant	Semantic tags (v0.1.0 to v0.6.0) maintained. Release commits detail functional additions and bug fixes.
CK-22	Scheduling Logic	Doctor schedule validation prevents concurrent double-booking of identical time slots.	backend/src/ modules/ appointments/ service.ts.	Compliant	Slot availability query checks for existing active appointments before slot reservation.
CK-23	Scheduling Logic	Doctor date-specific leave and holiday calendar integrated into appointment booking validation.	Appointment validation logic and Doctor schema.	MNC	System only validated weekly day-off recurring rules. Date-specific doctor leave calendar was missing, permitting invalid bookings.
CK-24	Billing Accuracy	Invoice calculations handle item additions, tax rates and medicine deductions with mathematical precision.	backend/src/ modules/billing/ calculator.ts.	Compliant	Pure calculator functions implement explicit financial rounding, itemized sub-totals and GST calculations.
CK-25	Billing Accuracy	Server-side printable PDF generation implemented for invoice receipts and patient ID cards.	backend/src/ services/ pdfService.ts and routes.	OFI	Initial implmentation relied on browser window.print(), causing inconsistent formatting across client devices.
CK-26	Error Handling	Global centralized error handling middleware catches unhandled rejections and formats error responses.	backend/src/ middleware/ errorHandler.ts.	Compliant	Custom AppError hierarchy maps validation, authentication and not-found exceptions to standard JSON responses.
CK-27	Deployment	Containerization configuration provides multi-stage builds and minimal attack	backend/Dockerfile and docker-compose.yml.	Compliant	Dockerfile uses node:20-alpine multi-stage build, running under non-root node user in

Item ID	Audit Area	Checkpoint / Evaluation Criteria	Evidence Examined	Status	Audit Observation and Finding
		surface.			production stage.
CK-28	Documentation	API endpoint contracts and data transfer objects documented with clear request and response examples.	docs/ and inline route annotations.	Compliant	Comprehensive API documentation and Postman collections maintained across all four core functional modules.

4. Audit Observations and Non-Conformance Report (NCR)

Our audit identified 6 findings across the 28 evaluated checkpoints: 2 Major Non-Conformances, 2 Minor Non-Conformances and 2 Opportunities for Improvement:

NCR ID	Type	Checkpoint	Affected Component	Description of Finding	Operational Risk and Impact	Owner	Target Date
NCR-01	MNC	CK-06	backend/ controllers/ and backend/server.js	Legacy JavaScript prototype files remained in root backend folder alongside modern TypeScript src/ directory.	Developers could inadvertently import or edit legacy files, causing dual-maintenance confusion and broken builds.	Urva Gandhi	Week 14 (48h)
NCR-02	MNC	CK-23	appointments/ service.ts and models/Doctor.ts	Doctor scheduling validated only recurring weekly days-off but lacked date-specific leave and emergency absence tracking.	Patients could successfully book and pay for appointments on days when the consulting physician was on planned leave.	Urva Gandhi	Week 14 (72h)
NCR-03	MiNC	CK-12	tests/setup.ts and backend/src/socket/	Jest test execution emitted open handle warnings due to unclosed Redis and Socket.io socket server listeners during teardown.	Automated CI/CD test runners hang intermittently or consume excessive memory, slowing deployment verification.	Urva Gandhi	Week 14 (48h)
NCR-04	MiNC	CK-20	backend/src/	Repository	New	Rakshit	Week 14

NCR ID	Type	Checkpoint	Affected Component	Description of Finding	Operational Risk and Impact	Owner	Target Date
			db/ and package.json	lacked an automated seed script for mock doctors, patients, appointment slots and admin accounts.	developer environments start with empty databases, hindering local testing and automated demonstration .	Gajnotar	(48h)
NCR-05	OFI	CK-13	patients/tests/	Patient module test coverage stood at 82.1%, slightly below the target project threshold of 85.0%.	Corner cases in patient phone number validation and duplicate medical record number checks could escape detection.	Rakshit Gajnotar	Week 15
NCR-06	OFI	CK-25	billing/ controller.ts and frontend Billing.tsx	Invoice printing relied on client-side CSS print media queries rather than cryptographic ally stable server-side PDF generation.	Invoice formatting varied across operating systems and browsers, creating unprofessiona l patient receipts.	Urva Gandhi	Week 15

5. Root Cause Analysis and Audit Metrics

We performed a 5-Whys root cause analysis on the two Major Non-Conformances to understand why these deficiences occured:

NCR Reference	Problem Statement	5-Whys Diagnostic Steps	Fundamental Root Cause	Preventive Safeguard
NCR-01 (MNC)	Legacy JS files remained in root backend directory.	1. Why? Developers left original prototype files intact. 2. Why? TypeScript rewrite was placed inside src/ to avoid breaking active scripts. 3. Why? Package.json main pointed to server.js. 4. Why? Build script had not been updated to	Absence of an explicit architectural clean-up step in sprint review checklists following major language migration.	Add an automated repository linting rule that flags any .js files located outside configuration files in the root folder.

NCR Reference	Problem Statement	5-Whys Diagnostic Steps	Fundamental Root Cause	Preventive Safeguard
		point to dist/server.js. 5. Why? No repository clean-up checklist existed prior to sprint freeze.		
NCR-02 (MNC)	Missing date-specific doctor leave validation.	1. Why? Patients could book slots on doctor leave dates. 2. Why? Booking query checked only weekly day-off array (e.g. Sunday). 3. Why? Doctor schema lacked an explicit leaves array with start and end dates. 4. Why? Initial requirement analysis focused on recurring outpatient clinics. 5. Why? Clinical exception scenarios were not detailed in early user stories.	Incomplete requirement elicitation regarding clinical exception workflows during sprint backlog creation.	Incorporate edge-case scenario walkthroughs (leaves, public holidays, emergency cancellations) during sprint backlog grooming.

Audit Compliance Metrics

The quantitative summary of our audit evaluation is presented below:

Audit Metric Indicator	Initial Count (Week 14 Audit)	Initial Percentage	Post-CAPA Count (Week 15)	Target Standard
Total Checkpoints Audited	28	100.0%	28	28 Checkpoints
Fully Compliant Checkpoints	22	78.57%	27	>= 90.0%
Major Non-Conformances (MNC)	2	7.14%	0	0 (Zero Tolerance)
Minor Non-Conformances (MiNC)	2	7.14%	0	0 at Release
Opportunities for Improvement (OFI)	2	7.14%	1	<= 3 Acceptable
Audit Compliance Index (ACI)	--	78.57%	96.43%	>= 95.0%

Our initial Audit Compliance Index of 78.57% showed that corrective work was needed before Milestone M6. Following the execution of our CAPA plan, compliance reached 96.43%, satisfying our pre-release quality gate.

6. Corrective and Preventive Action (CAPA) Plan

Every identified non-conformance was assigned an actionable remediation plan, an owner and a verification commit in the repository:

Action ID	NCR Ref	Corrective Action Implemented	Preventive Safeguard	Owner	Verification Commit	Status
CAPA-01	NCR-01	Safely deleted all legacy root backend/controllers/*.js, backend/server.js and obsolete JSX files via git rm.	Updated backend/package.json main entry to dist/server.js and configured tsconfig to disallow un-tracked JS.	Urva Gandhi	Commit e72547d verified clean build without legacy imports.	Verified and Closed
CAPA-02	NCR-02	Extended Doctor model with leaves schema; added validation in appointment booking service checking leave intervals.	Added automated integration test verifying HTTP 400 rejection when booking on approved leave dates.	Urva Gandhi	Verified in appointments.test.ts passing 18 of 18 test cases.	Verified and Closed
CAPA-03	NCR-03	Added explicit redisClient.quit() and socketServer.close() hooks inside afterAll teardown blocks across test files.	Configured jest.config.js with --detectOpenHandles flag to catch any asynchronous listener leaks in CI.	Urva Gandhi	Commit a02a897 confirmed 340 tests pass with zero open handles.	Verified and Closed
CAPA-04	NCR-04	Authored dedicated database seed script (backend/src/db/seed.ts) generating Indian doctors, sample patients and slots.	Added npm run seed script to package.json for one-command environment initialization.	Rakshit Gajnotar	Commit e94d0b6 added Dr. Vedant Mehta and seed data.	Verified and Closed
CAPA-05	NCR-05	Authored 6 additional unit tests covering patient phone formatting and edge-case duplicate medical record validation.	Enforced Jest coverage threshold of >= 85% in continuous integration configuration.	Rakshit Gajnotar	Patient module coverage reached 89.4%.	Verified and Closed
CAPA-06	NCR-06	Engineered server-side pdfService.ts using pdf-lib to render tamper-resistant PDF invoices and ID slips.	Exposed dedicated GET /api/v1/billing/:id/pdf and GET /api/v1/patients/:id/id-card	Urva Gandhi	Commit fd55fab added server-side PDF generator and tests.	Verified and Closed

Action ID	NCR Ref	Corrective Action Implemented	Preventive Safeguard	Owner	Verification Commit	Status
			authenticated endpoints.			

7. Conclusion

Conducting an internal software project audit at Milestone M5 provided concrete visibility into our Hospital Management System codebase and engineering processes. By evaluating 28 checkpoints across architecture, code quality, security and testing, we caught critical gaps before final delivery. Removing legacy JavaScript prototype files restored architectural clarity, while adding doctor leave validation protected clinical scheduling accuracy. With all CAPA actions verified and our compliance index rising to 96.43%, the project is approved to proceed to Milestone M6 final release.